

Quiz Questions:

Question 1

A system processes various types of financial transactions (e.g., deposits, withdrawals, transfers). Each transaction type requires specific validation and processing logic, but they all share a common interface for initiation and status tracking. The system is designed to allow new transaction types to be added without modifying the core transaction processing engine. Which design principle and OOP concept combination primarily enables this flexibility and extensibility?

A Encapsulation and Data Hiding

B Composition and Aggregation

C Abstraction and Runtime Polymorphism

D Static Members and Object Identity

Question 2

Consider a Vehicle class that can be either a Car or a Motorcycle. Both Car and Motorcycle have distinct methods for calculating fuel efficiency. A function print_vehicle_details accepts a Vehicle object and calls a calculate_fuel_efficiency method on it, without knowing the specific type of vehicle at compile time. Which of the following OOP concepts are directly demonstrated in this scenario?

A Method Overloading

B Runtime Polymorphism

C Inheritance

D Dynamic Dispatch

E Object Identity

Question 3

A Book object in a library system needs to track its availability. When a member borrows the book, its status changes from 'Available' to 'Borrowed'. When returned, it changes back to 'Available'. A 'Borrowed' book cannot be borrowed again until it's returned. Which UML behavioral model is best suited to represent these distinct states and the transitions between them, along with the conditions for valid state changes?

A Sequence Diagram

B Activity Diagram

C Use Case Diagram

D State Diagram

Question 4

A Customer can place multiple Orders, but an Order is always placed by exactly one Customer. An Order cannot exist without being associated with a Customer. If a Customer is removed from the system, all their Orders should also be removed. Which UML relationship types are most appropriate for modeling the strong lifecycle dependency between Customer and Order in this scenario?

A Association

B Aggregation

C Composition

D Dependency

E Generalization

Question 5

A FoodOrder object encapsulates its LinelItems. Each LinelItem represents a specific food item with a quantity. The LinelItems are created and managed exclusively by the FoodOrder object and cease to exist if the FoodOrder is cancelled or completed. No LinelItem can exist without being part of a FoodOrder. Which type of relationship best describes the connection between FoodOrder and LinelItem?

A Association

B Aggregation

C Dependency

D Composition

Question 6

A PaymentProcessor module needs to handle various payment methods (e.g., Credit Card, PayPal, Bank Transfer). The PaymentProcessor should interact with these methods through a common IPaymentMethod interface. When a new payment method is introduced, the PaymentProcessor's code should not need modification. Which OOP principle is most directly applied here to achieve extensibility?

A Encapsulation

B Inheritance

C Abstraction

D Cohesion

Question 7

During an online examination, a student starts an attempt, answers questions, and finally submits. After submission, the answers cannot be changed, and the attempt is evaluated. The flow of these actions (start, answer, submit, evaluate) and the sequence of interactions between the Student, Examination, and Submission objects are crucial. Which UML diagram type is best suited to visualize this step-by-step interaction and message flow?

A Class Diagram

B Activity Diagram

C Sequence Diagram

D Component Diagram

Question 8

An Employee object has an annual_salary attribute. The business rule states that the annual_salary cannot be set to a negative value. A set_salary method is used to update this attribute, which includes a validation check to ensure the salary is non-negative before updating the internal state. Which of the following OOP concepts are effectively demonstrated by this design?

A Data Hiding

B Encapsulation

C Inheritance

D Maintaining Valid Object State

E Abstraction

Question 9

A Logger interface defines a log_message(message: str) method. Concrete implementations include FileLogger, DatabaseLogger, and ConsoleLogger. A client application needs to log messages but should not depend on which specific logger implementation is being used. This approach allows for swapping logger implementations without altering the client code. What is the UML relationship between the FileLogger class and the Logger interface?

A Generalization

B Association

C Aggregation

D Realization

Question 10

A Course can have a maximum of 50 Students. A Student can enroll in multiple Courses. What is the correct multiplicity representation for the relationship between Course and Student from the perspective of Course to Student and Student to Course?

A Course 1 --- 1..* Student, Student 1 --- 1..* Course

B Course 0..* --- 1 Student, Student 0..* --- 1 Course

C Course 1 --- 0..50 Student, Student 1 --- 0..* Course

D Course 1 --- * Student, Student 1 --- * Course

Concept Questions

Question 1:

In a ShoppingCart object, the total_price should always be the sum of the prices of all LineItems. If an external operation could directly modify total_price without updating LineItems, the object's state could become inconsistent. Explain how the concept of 'invariants' relates to maintaining a valid object state within a ShoppingCart. Discuss how proper encapsulation and controlled modification of state, typically through methods, enforce these invariants and prevent invalid states.

Answer: An invariant is a condition that must remain true throughout the lifetime of an object. In a ShoppingCart, the invariant is that total price must always equal the sum of the prices of all LineItems. Encapsulation protects the internal data of the ShoppingCart by making attributes private and allowing modifications only through controlled methods. For example, addItem(), remove_item() and calculate_total() can manage the cart contents and maintain the correct total. The total_price should not be directly modifiable by external code. Instead, the total should be recalculated whenever an item is added or removed. This prevents inconsistent states and ensures that the ShoppingCart always maintains its invariant.

Question 2:

A SmartDevice can have optional capabilities like Camera, GPS, or Bluetooth. If every capability were modeled using inheritance (e.g., SmartDeviceWithCamera, SmartDeviceWithGPS), it could lead to an explosion of classes and a rigid hierarchy, making it hard to manage combinations of capabilities (e.g., a device with both camera and GPS). Explain why composition might be a more suitable approach than inheritance for modeling such optional capabilities. Use the SmartDevice scenario to illustrate how composition provides greater flexibility and extensibility when dealing with varied combinations of features.

Answer: Composition means building a class by combining objects that provide different functionalities. Inheritance creates a parent-child relationship in which a child class inherits properties and methods from a parent class. If every SmartDevice capability were implemented using inheritance, classes such as SmartDeviceWithCamera, SmartDeviceWithGPS, and SmartDeviceWithCameraAndGPS would be required. This could result in a large number of classes and a rigid hierarchy. Using composition, a SmartDevice can contain separate Camera, GPS, and Bluetooth components. Each device can include whichever components it requires. For example, one device can contain Camera and GPS, while another contains only Bluetooth. Composition is more flexible because capabilities can be combined, replaced, or extended independently without creating a new subclass for every combination. Therefore, composition is more suitable for modeling optional SmartDevice capabilities.

Question 3:

During a customer checkout process in an e-commerce system, a CheckoutManager object initiates a PaymentGateway interaction, which in turn communicates with an external BankService. The BankService returns a result to the PaymentGateway, and finally, the PaymentGateway relays the payment status back to the CheckoutManager. Describe how UML Sequence Diagram reasoning would represent this flow of interactions. Specifically, explain how message ordering, object lifelines, and return messages are depicted, and why this diagram is effective for understanding dynamic object collaboration in this scenario.

Answer: A UML Sequence Diagram represents interactions between objects in chronological order.

In the e-commerce checkout scenario, the diagram contains three object lifelines:

- CheckoutManager
- PaymentGateway
- BankService

The interaction proceeds as follows:

1. CheckoutManager sends a payment request to PaymentGateway.
2. PaymentGateway sends a payment authorization request to BankService.
3. BankService processes the request and returns the payment result to PaymentGateway.
4. PaymentGateway returns the payment status to CheckoutManager.

Each object is represented by a lifeline, and horizontal arrows represent messages exchanged between objects. Time progresses from top to bottom. Return messages show the results sent back to the calling object.

The sequence diagram helps developers understand the order of operations, responsibilities of each object, and dependencies between components. It is useful for understanding and debugging dynamic object collaboration.

Question 4:

Consider a NotificationService that can send messages using various mechanisms such as email, SMS, or push notifications. A client application needs to send a notification but should not be aware of the specific underlying mechanism. Explain how the OOP concept of abstraction, specifically through interfaces or abstract classes, allows the NotificationService to be highly extensible. Discuss how this design makes it possible to introduce new notification mechanisms (e.g., a new messaging app integration) without requiring any changes to the client's code, demonstrating the benefits of dependency on abstractions.

Answer: Abstraction hides implementation details and exposes only the essential operations that a client needs to use. A NotificationService can define a common interface such as NotificationSender, containing a method called sendNotification(). Classes such as EmailNotification, SMSNotification, and PushNotification implement this interface using their own delivery mechanisms. The client application interacts with NotificationService through the common interface without needing to know how a particular notification is delivered. When a new messaging mechanism is introduced, a new class can implement the same interface. The NotificationService can use that implementation without changing the client's code, provided the new implementation is registered or configured appropriately. This approach demonstrates dependency on abstractions: high-level code depends on a stable interface rather than concrete implementations. It improves flexibility, extensibility, and maintainability.

Question 5:

A University contains Departments, and each Department has Professors. If a University ceases to exist, its Departments also cease to exist, but the Professors might continue their careers at other institutions. Explain the difference between Aggregation and Composition UML relationships using this University, Department, and Professor scenario. Clearly articulate the concepts of object ownership and lifecycle dependency that differentiate these two types of 'whole-part' relationships and justify which one applies where.

Answer: Composition is a strong whole-part relationship in which the part's lifecycle depends on the whole. Aggregation is a weaker whole-part relationship in which the part can exist independently of the whole.

In the given scenario:

- University and Department — Composition: A Department belongs to a University, and according to the question, the Departments cease to exist when the University ceases to exist.
- Department and Professor — Aggregation: Professors may continue their careers at other institutions even if their Department or University ceases to exist. Their lifecycles are independent of the institution.

In UML, composition is represented by a filled diamond at the whole end of the relationship, while aggregation is represented by a hollow diamond. Therefore, composition is appropriate for University–Department, while aggregation is appropriate for Department–Professor, based on the stated lifecycle conditions.